

Name: Jiya Yogesh Varma

PRN: 202501050023

Division: Chemical

Batch: CH2

Course: EDS





Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

Sample Test Cases

Test case 1

1 2 3 4 5 6 7 8 9 10

Mean of even and odd numbers:

Odd : 5.0

Even : 6.0

dtype: float64

Test case 2

4 4 4 6 6 2 2 2 10 12 14 16

Mean of even and odd numbers:

Even : 6.833333

dtype: float64

Test case 3

1 5 7 9 11 13 15 17 111 21 25

Mean of even and odd numbers:

Odd : 21.363636

dtype: float64

```
1 import pandas as pd
2
3 # Take inputs from the user to
4 # create a list of numbers
5 numbers = list(map(int,
6 input().split()))
7
8 # Create a Pandas series from
9 # the list of numbers
10 series = pd.Series(numbers)
11
12 # Grouping by even and odd
13 # numbers and calculating the
14 # mean
15 grouped = series.groupby(series
16 % 2 == 0).mean()
17
18 # Display the mean of even and
19 # odd numbers with labels
20 grouped.index = ['Even' if
21 is_even else 'Odd' for is_even
22 in grouped.index]
23 print("Mean of even and odd
24 numbers:")
25 print(grouped)
```

Average time

0.032 s

32.00 ms

Maximum time

0.064 s

64.00 ms

3 out of 3 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 64 ms

Debug

Expected output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd num  
bers:

Odd : 5.0

Actual output

1 2 3 4 5 6 7 8 9 10

Mean of even and odd num  
bers:

Odd : 5.0





4.1.2. Dictionary to dataframe

41:41



A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

Sample Test Cases

Test case 1

Original DataFrame:

|   | Name    | Age |
|---|---------|-----|
| 0 | Alice   | 25  |
| 1 | Bob     | 30  |
| 2 | Charlie | 35  |

New name: Susan

New age: 29

After adding a row:

|   | Name    | Age |
|---|---------|-----|
| 0 | Alice   | 25  |
| 1 | Bob     | 30  |
| 2 | Charlie | 35  |
| 3 | Susan   | 29  |

Index of row to modify: 0

```
1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie'],
6     'Age': [25, 30, 35],
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
12 # Display the original DataFrame
13 print("Original DataFrame:")
14 print(df)
15
16 # Adding a new row
17 new_name = input("New name: ")
18 new_age = int(input("New age: "))
19 df.loc[len(df)] = [new_name, new_age]
20
21 # Display the DataFrame after adding a new row
22 print("After adding a row:\n",df)
23
24 # Modifying a row
25 row_to_modify = int(input("Index of row to modify: "))
26 new_age_value = int(input("New age: "))
27 df.at[row_to_modify, "Age"] = new_age_value
28
29 # Display the DataFrame after modifying a row
30 print("After modifying a row:")
31 print(df)
32
33 # Deleting a row
34 row_to_delete = int(input("Index of row to delete: "))
35 df = df.drop(index=row_to_delete).reset_index(drop=True)
36
```

|              |           |              |           |
|--------------|-----------|--------------|-----------|
| Average time | 0.282 s   | Maximum time | 0.326 s   |
|              | 282.00 ms |              | 326.00 ms |

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

| Test case 1         | 326 ms              | Debug |  |  |
|---------------------|---------------------|-------|--|--|
| Expected output     | Actual output       |       |  |  |
| Original DataFrame: | Original DataFrame: |       |  |  |
| .....Name.....Age   | .....Name.....Age   |       |  |  |
| 0.....Alice.....25  | 0.....Alice.....25  |       |  |  |





4.1.3. Student Information

21:42

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

**Note:**  
Refer to the displayed test cases for better understanding.

Sample Test Cases

Test case 1

studentdata.txt

First five rows:

Name Age Grade

0 John 25 A

1 Alin 22 B

2 Emma 24 A

3 Mike 23 C

4 Sony 21 B

Average age: 23.29

Students with a grade up to B

Name Age Grade

0 John 25 A

1 Alin 22 B

2 Emma 24 A

4 Sony 21 B

5 Amia 26 A

```
1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file,
6 sep="\s+", header=None, names=
7 ["Name", "Age", "Grade"])
8
9 # write your code here..
10 print("First five rows:")
11 print(data.head())
12
13 average_age=round(data["Age"].mean(), 2)
14 print(f"Average age: {average_age}")
15
16 filtered_students =
17 data[data["Grade"] <= "B"]
18 print("Students with a grade up to B")
19 print(filtered_students)
```

Average time

0.124 s

124.00 ms

Maximum time

0.161 s

161.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 161 ms

Debug

Expected output

studentdata.txt

First five rows:

Name Age Grade

Actual output

studentdata.txt

First five rows:

Name Age Grade



Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

| Date       | Product   | Quantity | Price | City        |
|------------|-----------|----------|-------|-------------|
| 2025-01-01 | Product A | 5        | 20    | New York    |
| 2025-01-01 | Product B | 3        | 15    | Los Angeles |
| 2025-01-02 | Product A | 7        | 20    | New York    |
| 2025-01-02 | Product C | 4        | 30    | Chicago     |
| 2025-01-03 | Product B | 2        | 15    | Chicago     |
| 2025-01-03 | Product A | 8        | 20    | Los Angeles |
| 2025-01-04 | Product C | 6        | 30    | New York    |
| 2025-01-04 | Product B | 5        | 15    | Los Angeles |
| 2025-01-05 | Product A | 3        | 20    | Chicago     |
| 2025-01-05 | Product C | 10       | 30    | Los Angeles |

**Note:**  
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test case 1

sales\_data.csv

Best month: 2025-01

Total sales: \$1210.00

monthForS...

Submit

1import pandas as pd

2

3# Prompt the user for the file name

4file\_name = input()

5

6# Load the data

7df = pd.read\_csv(file\_name)

8

9# Find the month with the highest total sales

10df['Date'] =

11pd.to\_datetime(df["Date"])

12df["Month"] =

13df["Date"].dt.strftime("%Y-%m")

14df["Total

15Sales"]=df["Quantity"] \*

16df["Price"]

17

18# Find the month with the highest total sales

19sales\_by\_month=df.groupby("Mont

20h")["Total Sales"].sum()

21best\_month =

22sales\_by\_month.idxmax()

23highest\_sales =

24sales\_by\_month.max()

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

Average time0.079 s79.00 ms

Maximum time0.143 s143.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1143 ms

Debug

Expected output

Actual output

sales\_data.csvsales\_data.csv

Best month: 2025-01Best month: 2025-01

Total sales: \$1210.00Total sales: \$1210.00

< Prev

Reset

Submit

Next >





4.2.2. Best Selling Product

00:18

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

**Note:**  
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test case 1

sales\_data.csv

Best selling product: Product A

Total quantity sold: 23

```
1 import pandas as pd
2
3 # Prompt the user for the file
  name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8 product_sales =
  df.groupby("Product")
  ["Quantity"].sum()
9 # Find the product with the
  highest total quantity sold
10 best_product =
  product_sales.idxmax()
11 highest_quantity =
  product_sales.max()
12
13 # Find the product with the
  highest total quantity sold
14
15
16 # Display the result
17 print(f"Best selling product:
  {best_product}")
18 print(f"Total quantity sold:
  {highest_quantity}")
19
```

Average time  
0.059 s  
59.33 ms

Maximum time  
0.105 s  
105.00 ms

1 out of 1 shown test case(s) passed  
2 out of 2 hidden test case(s) passed

| Expected output                 | Actual output                   |
|---------------------------------|---------------------------------|
| sales_data.csv                  | sales_data.csv                  |
| Best selling product: Product A | Best selling product: Product A |
| Total quantity sold: 23         | Total quantity sold: 23         |





4.2.3. City that Sold the Most Products 00:12

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date , Product , Quantity , Price , and City .
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

**Note:**  
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test case 1

sales\_data.csv

City sold the most products: Los Angeles

```
1 import pandas as pd
2
3 # Prompt the user for the file
  name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # write the code..
10 # write the code..
11 city_sales = df.groupby("City")
   ["Quantity"].sum()
12 best_city=city_sales.idxmax()
13 # Display the result
14 print(f"City sold the most
   products: {best_city}")
15
```

|              |              |
|--------------|--------------|
| Average time | Maximum time |
| 0.058 s      | 0.107 s      |
| 58.00 ms     | 107.00 ms    |

- ✓ 1 out of 1 shown test case(s) passed
- ✓ 2 out of 2 hidden test case(s) passed

|                          |                          |  |  |  |
|--------------------------|--------------------------|--|--|--|
| ✓ Test case 1 107 ms     | Debug                    |  |  |  |
| Expected output          | Actual output            |  |  |  |
| sales_data.csv           | sales_data.csv           |  |  |  |
| City sold the most produ | City sold the most produ |  |  |  |
| cts: Los Angeles         | cts: Los Angeles         |  |  |  |





4.2.4. Most Frequently Sold Product Pairs

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

| Date       | Product   | Quantity | Price | City        |
|------------|-----------|----------|-------|-------------|
| 2025-01-01 | Product A | 5        | 20    | New York    |
| 2025-01-01 | Product B | 3        | 15    | Los Angeles |
| 2025-01-02 | Product A | 7        | 20    | New York    |
| 2025-01-02 | Product C | 4        | 30    | Chicago     |
| 2025-01-03 | Product B | 2        | 15    | Chicago     |
| 2025-01-03 | Product A | 8        | 20    | Los Angeles |
| 2025-01-04 | Product C | 6        | 30    | New York    |
| 2025-01-04 | Product B | 5        | 15    | Los Angeles |
| 2025-01-05 | Product A | 3        | 20    | Chicago     |
| 2025-01-05 | Product C | 10       | 30    | Los Angeles |

Explanation:

Transactions:

- **2025-01-01:** Product A, Product B
- **2025-01-02:** Product A, Product C
- **2025-01-03:** Product B, Product A
- **2025-01-04:** Product C, Product B
- **2025-01-05:** Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B:** Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C:** Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C:** Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- **Product A and Product B** (2 times)
- **Product A and Product C** (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

Test case 1

|                                  |  |
|----------------------------------|--|
| sales_data.csv                   |  |
| Product A and Product B: 2 times |  |
| Product A and Product C: 2 times |  |

```
1 import pandas as pd
2 from itertools import
  combinations
3 from collections import
  Counter
4
5 # Prompt user to input the
  file name
6 file_name = input()
7
8 # Read data from the
  specified CSV file
9 df = pd.read_csv(file_name)
10
11 # write the code
12
13 # write the code
14 date_products = {}
15
16 for date, group in
  df.groupby('Date'):
17     products =
18     group['Product'].unique()
19     if len(products) > 1:
20         date_products[date] =
21         products
22
23     # Count product pairs
24     pair_counter = Counter()
25
26     for products in
27     date_products.values():
28         # Sort to avoid duplicate
29         pairs like (A, B) and (B, A)
30         pairs =
31         combinations(sorted(products),
32         2)
33         pair_counter.update(pairs)
34
35 # Find the maximum frequency
36 if pair_counter:
37     max_count =
38     max(pair_counter.values())
39
40 # Output the most
41 frequent product pairs
42 for pair, count in
43     pair_counter.items():
44     if count == max_count:
45         print(f"{pair[0]}
46         and {pair[1]}: {count} times")
47     else:
48         print("No product pairs")
```

|              |              |
|--------------|--------------|
| Average time | Maximum time |
| 0.062 s      | 0.114 s      |
| 62.00 ms     | 114.00 ms    |

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

|                                  |                                  |       |  |  |
|----------------------------------|----------------------------------|-------|--|--|
| Test case 1                      | 114 ms                           | Debug |  |  |
| Expected output                  | Actual output                    |       |  |  |
| sales_data.csv                   | sales_data.csv                   |       |  |  |
| Product A and Product B: 2 times | Product A and Product B: 2 times |       |  |  |
| Product A and Product C: 2 times | Product A and Product C: 2 times |       |  |  |



You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

| P<br>a<br>s<br>s<br>e<br>n<br>g<br>e<br>r<br>I<br>d | S<br>u<br>r<br>v<br>i<br>v<br>e<br>d | P<br>c<br>l<br>a<br>s<br>s | N<br>a<br>m<br>e | S<br>e<br>x | A<br>g<br>e | S<br>i<br>b<br>S<br>p | P<br>a<br>r<br>c<br>h | T<br>i<br>c<br>k<br>e<br>t | F<br>a<br>r<br>e | C<br>a<br>b<br>i<br>n | E<br>m<br>b<br>a<br>r<br>k<br>e<br>d |
|-----------------------------------------------------|--------------------------------------|----------------------------|------------------|-------------|-------------|-----------------------|-----------------------|----------------------------|------------------|-----------------------|--------------------------------------|
|                                                     |                                      |                            |                  |             |             |                       |                       |                            |                  |                       |                                      |

Sample Data:

| PassengerId | Survived | Pclass | Name                                              | Sex    | Age | SibSp | Parch | Ti               |
|-------------|----------|--------|---------------------------------------------------|--------|-----|-------|-------|------------------|
| 1           | 0        | 3      | "Braund, Mr. Owen Harris"                         | male   | 22  | 1     | 0     | A/5 21171,7      |
| 2           | 1        | 1      | "Cumings, Mrs. John Bradley (Florence Briggs Thay |        |     |       |       |                  |
| 3           | 1        | 3      | "Heikkinen, Miss. Laina"                          | female | 26  | 0     | 0     | STON/O2. 3       |
| 4           | 1        | 1      | "Futrelle, Mrs. Jacques Heath (Lily May Peel)"    | fe     |     |       |       |                  |
| 5           | 0        | 3      | "Allen, Mr. William Henry"                        | male   | 35  | 0     | 0     | 373450,8.0       |
| 6           | 0        | 3      | "Moran, Mr. James"                                | male   |     | 0     | 0     | 330877,8.4583,,Q |
| 7           | 0        | 1      | "McCarthy, Mr. Timothy J"                         | male   | 54  | 0     | 0     | 17463,51.86      |
| 8           | 0        | 3      | "Palsson, Master. Gosta Leonard"                  | male   | 2   | 3     | 1     | 34990            |
| 9           | 1        | 3      | "Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg |        |     |       |       |                  |
| 10          | 1        | 2      | "Nasser, Mrs. Nicholas (Adele Achem)"             | female | 14  |       |       |                  |

**Note:** Refer to the visible test case for better reference.

Sample Test Cases

Test case 1

|             |          |        |      |         |          |
|-------------|----------|--------|------|---------|----------|
| PassengerId | Survived | Pclass | Fare | Cabin   | Embarked |
| 0           | 1        | 0      | 3    | 7.2500  | NaN      |
| 1           | 2        | 1      | 1    | 71.2833 | C85      |
| 2           | 3        | 1      | 3    | 7.9250  | NaN      |

titanicData... Submit

Explorer

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # 1. Display the first 5 rows of the dataset
8 print(data.head())
9
10 # 2. Display the last 5 rows of the dataset
11 print(data.tail())
12
13 # 3. Get the shape of the dataset (number of rows and columns)
14 print(data.shape)
15
16 # 4. Get a summary of the dataset
17 print(data.info())
18
19 # 5. Get basic statistics of the dataset
20 print(data.describe())
21
22 # 6. Check for missing values and display the count
23 print(data.isnull().sum())
24 # 7. Fill missing values in the 'Age' column with the median age
25
26 data['Age'].fillna(data['Age'].median(), inplace=True)
27
28 # 8. Fill missing values in the 'Embarked' column with the most frequent value (mode)
29
30 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
31
32 # 9. Drop the 'Cabin' column due to many missing values
33 data.drop(columns=['Cabin'], inplace=True)
34
35 # 10. Create a new column 'FamilySize' by adding
```

Debugger

Average time0.562 s562.00 ms

Maximum time0.562 s562.00 ms

1 out of 1 shown test case(s) passed

Test case 1562 ms

Debug

Expected output

Actual output

|             |          |        |      |        |          |
|-------------|----------|--------|------|--------|----------|
| PassengerId | Survived | Pclass | Fare | Cabin  | Embarked |
| 0           | 1        | 0      | 3    | 7.2500 | NaN      |

|             |          |        |      |        |          |
|-------------|----------|--------|------|--------|----------|
| PassengerId | Survived | Pclass | Fare | Cabin  | Embarked |
| 0           | 1        | 0      | 3    | 7.2500 | NaN      |

< Prev

Reset

Submit

Next >



You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

- 
- Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
  - Convert the 'Sex' column to numeric values (male: 0, female: 1).
  - One-hot encode the 'Embarked' column, dropping the first category.
  - Get the mean age of passengers.
  - Get the median fare of passengers.
  - Get the number of passengers by class.
  - Get the number of passengers by gender.
  - Get the number of passengers by survival status.
  - Calculate the survival rate of passengers.
  - Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

| P<br>a<br>s<br>s<br>e<br>n<br>g<br>e<br>r<br>I<br>d | S<br>u<br>r<br>v<br>i<br>v<br>e<br>d | P<br>c<br>l<br>a<br>s<br>s | N<br>a<br>m<br>e | S<br>e<br>x | A<br>g<br>e | S<br>i<br>b<br>S<br>p | P<br>a<br>r<br>c<br>h | T<br>i<br>c<br>k<br>e<br>t | F<br>a<br>r<br>e | C<br>a<br>b<br>i<br>n | E<br>m<br>b<br>a<br>r<br>k<br>e<br>d |
|-----------------------------------------------------|--------------------------------------|----------------------------|------------------|-------------|-------------|-----------------------|-----------------------|----------------------------|------------------|-----------------------|--------------------------------------|
|                                                     |                                      |                            |                  |             |             |                       |                       |                            |                  |                       |                                      |

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ti
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thay
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",fe
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.0
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.86
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,34990
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,
```

**Note:** Refer to the visible test case for better reference.

Sample Test Cases—

Test case 1

29.69911764705882

14.4542

3...491

1...216

2...184

Name: Pclass, dtype: int64

0...577

1...314

Name: Sex, dtype: int64

0...540

Explorer

titanicData...

Submit

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-
6 Dataset.csv')
7 data['FamilySize'] =
8 data['SibSp'] + data['Parch']
9
10
11 # 1. Create a new column
12 'IsAlone' (1 if alone, 0
13 otherwise)
14 data['IsAlone'] =
15 np.where(data['FamilySize']
16 == 0, 1, 0)
17
18 # 2. Convert 'Sex' to numeric
19 (male: 0, female: 1)
20 data['Sex'] =
21 data['Sex'].map({'male': 0,
22 'female': 1})
23
24 # 3. One-hot encode the
25 'Embarked' column
26 data = pd.get_dummies(data,
27 columns=['Embarked'])
28
29 # 4. Get the mean age of
30 passengers
31 mean_age = data['Age'].mean()
32 print(mean_age)
33
34 # 5. Get the median fare of
35 passengers
36 median_fare =
37 data['Fare'].median()
38 print(median_fare)
39
40 # 6. Get the number of
41 passengers by class
42 print(
43 data['Pclass'].value_counts())
44
45 # 7. Get the number of
46 passengers by gender
47 print(
48 data['Sex'].value_counts())
49
50 # 8. Get the number of
51 passengers by survival status
52 print(
53 data['Survived'].value_counts(
54 ))
55
56 # 9. Calculate the overall
```

Average time0.977 s977.00 ms

Maximum time0.977 s977.00 ms

1 out of 1 shown test case(s) passed

Test case 1977 ms

Debug

| Expected output   | Actual output     |
|-------------------|-------------------|
| 29.69911764705882 | 29.69911764705882 |
| 14.4542           | 14.4542           |
| 3...491           | 3...491           |
| 1...216           | 1...216           |
| 2...184           | 2...184           |





You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked\_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

| P | S | P | N | S | A | S | P | T | F | C | E |
|---|---|---|---|---|---|---|---|---|---|---|---|
| a | u | c | a | e | g | i | a | i | a | a | m |
| s | r | l | m | x | e | b | r | c | r | b | a |
| s | v | a | e |   |   | s | c | k | e | i | r |
| e | i | s |   |   |   | p | h | e |   | n | k |
| r | v |   |   |   |   |   |   |   |   |   | e |
| i | e |   |   |   |   |   |   |   |   |   | d |
| d | d |   |   |   |   |   |   |   |   |   |   |

Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ti
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thay
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",fe
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.0
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.86
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,34990
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,
```

Note: Refer to the visible test case for better reference.

Sample Test Cases

Test case 1

Pclass

10.629630

20.472826

30.242363

Name: Survived, dtype: float64

Embarked\_S

titanicData... Submit

Explorer

```
data = pd.get_dummies(data,
columns=['Embarked'],
drop_first=True)

# 1. Calculate the survival
rate by class
print(data.groupby('Pclass')
['Survived'].mean())

#2. Calculate the survival
rate by embarked location
(Embarked_S)

print(data.groupby
('Embarked_S')
['Survived'].mean())

#3. Calculate the survival
rate by family size

print(data.groupby('FamilySize
') ['Survived'].mean())

#4. Calculate the survival
rate by being alone

print(data.groupby('IsAlone')
['Survived'].mean())

#5. Get the average fare by
class
print(data.groupby('Pclass')
['Fare'].mean())

#6. Get the average age by
class

print(data.groupby('Pclass')
['Age'].mean())

#7. Get the average age by
survival status

print(data.groupby('Survived')
['Age'].mean())

#8. Get the average fare by
survival status

print(data.groupby('Survived')
['Fare'].mean())

#9. Get the number of
survivors by class (sort by
values descending)
```

Debugger

Average time0.400 s400.00 ms

Maximum time0.400 s400.00 ms

1 out of 1 shown test case(s) passed

Test case 1400 ms

Debug

Expected output

Actual output

Pclass

Pclass

10.629630

10.629630

20.472826

20.472826

30.242363

30.242363

Name: Survived, dtype: fl

Name: Survived, dtype: f





You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

- 1. Get the number of survivors by gender (Sex).
- 2. Get the number of non-survivors by gender (Sex).
- 3. Get the number of survivors by embarkation location (Embarked\_S).
- 4. Get the number of non-survivors by embarkation location (Embarked\_S).
- 5. Calculate the percentage of children (Age < 18) who survived.
- 6. Calculate the percentage of adults (Age >= 18) who survived.
- 7. Get the median age of survivors.
- 8. Get the median age of non-survivors.
- 9. Get the median fare of survivors.
- 10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

| P | S | P | N | S | A | S | P | T | F | C | E |
|---|---|---|---|---|---|---|---|---|---|---|---|
| a | u | l | a | e | g | i | a | i | a | a | m |
| s | r | a | m | x | e | b | r | c | r | b | a |
| s | v | s | e |   |   | s | c | k | e | i | r |
| e | i |   |   |   |   | p | h | e |   | n | k |
| r | v |   |   |   |   |   |   |   |   |   |   |
| d | d | s |   |   |   |   |   |   |   |   | d |

Sample Data:

| PassengerId | Survived | Pclass | Name                                              | Sex    | Age | SibSp | Parch | Ti               |
|-------------|----------|--------|---------------------------------------------------|--------|-----|-------|-------|------------------|
| 1           | 0        | 3      | "Braund, Mr. Owen Harris"                         | male   | 22  | 1     | 0     | A/5 21171,7      |
| 2           | 1        | 1      | "Cumings, Mrs. John Bradley (Florence Briggs Thay |        |     |       |       |                  |
| 3           | 1        | 3      | "Heikkinen, Miss. Laina"                          | female | 26  | 0     | 0     | STON/O2. 3       |
| 4           | 1        | 1      | "Futrelle, Mrs. Jacques Heath (Lily May Peel)"    | fe     |     |       |       |                  |
| 5           | 0        | 3      | "Allen, Mr. William Henry"                        | male   | 35  | 0     | 0     | 373450,8.0       |
| 6           | 0        | 3      | "Moran, Mr. James"                                | male   |     | 0     | 0     | 330877,8.4583,,Q |
| 7           | 0        | 1      | "McCarthy, Mr. Timothy J"                         | male   | 54  | 0     | 0     | 17463,51.86      |
| 8           | 0        | 3      | "Palsson, Master. Gosta Leonard"                  | male   | 2   | 3     | 1     | 34990            |
| 9           | 1        | 3      | "Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg |        |     |       |       |                  |
| 10          | 1        | 2      | "Nasser, Mrs. Nicholas (Adele Achem)"             | female | 14  |       |       |                  |

Note: Refer to the visible test case for better reference.

Sample Test Cases

Test case 1

female...233

male...109

Name: Sex, dtype: int64

male...468

female...81

Name: Sex, dtype: int64

1...217

0...125

Name: Embarked\_S, dtype: int64

Explorer titanicData... Submit

1 import pandas as pd

2 import numpy as np

3

4 # Load the Titanic dataset

5 data = pd.read\_csv('Titanic-Dataset.csv')

6 data = pd.get\_dummies(data, columns=['Embarked'], drop\_first=True)

7

8 # 1. Get the number of survivors by gender

9 survivors\_by\_gender = data[data['Survived'] == 1]['Sex'].value\_counts()

10 print(survivors\_by\_gender)

11

12 # 2. Get the number of non-survivors by gender

13 non\_survivors\_by\_gender = data[data['Survived'] == 0]['Sex'].value\_counts()

14 print(non\_survivors\_by\_gender)

15 # 3. Get the number of survivors by embarked location (Embarked\_S)

16 survivors\_by\_embarked\_s = data[data['Survived'] == 1]['Embarked\_S'].value\_counts()

17 print(survivors\_by\_embarked\_s)

18

19 # 4. Get the number of non-survivors by embarked location (Embarked\_S)

20 non\_survivors\_by\_embarked\_s = data[data['Survived'] == 0]['Embarked\_S'].value\_counts()

21 print(non\_survivors\_by\_embarked\_s)

22

23 # 5. Percentage of children (Age < 18) who survived

24 children = data[data['Age'] < 18]

25 children\_survival\_rate = children['Survived'].mean()

26 print(children\_survival\_rate)

27 # 6. Percentage of adults (Age >= 18) who survived

28 adults = data[data['Age'] >= 18]

29 adults\_survival\_rate = adults['Survived'].mean()

Average time 0.254 s 254.00 ms

Maximum time 0.254 s 254.00 ms

1 out of 1 shown test case(s) passed

Test case 1 254 ms Debug

Expected output

female...233

male...109

Name: Sex, dtype: int64

male...468

female...81

Actual output

female...233

male...109

Name: Sex, dtype: int64

male...468

female...81